



Duale Hochschule Baden-Württemberg Mannheim

Projektarbeit

Rekurrente Neuronale Netze

Studiengang Wirtschaftsinformatik

Studienrichtung Data Science und Künstliche Intelligenz

Verfasser(in):	Luke Vierfuß und Sebastian Höfling
Matrikelnummer:	1091660 und 6043131
Firma:	Fresenius SE & Co. KGaA. und DATEV eG
Abteilung:	-
Kurs:	WDSKI25A
Studiengangsleiter:	Prof. Dr.-Ing. Pfisterer
Wissenschaftliche(r) Betreuer(in):	-
Firmenbetreuer(in):	-
Bearbeitungszeitraum:	04.05.2026 – 23.05.2026

Ehrenwörtliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Titel "*Rekurrente Neuronale Netze*" selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ort, Datum

Luke Vierfuß und Sebastian Höfling

Sperrvermerk

Ein Sperrvermerk sollte nur bei berechtigtem Bedarf gesetzt werden!

Beachten Sie, dass mit Sperrvermerk versehene Arbeiten nicht für weitere wissenschaftliche Zwecke außerhalb des Firmenkontextes oder zur Publikation verwendet werden dürfen.

Wir empfehlen, wenn möglich, auf den Sperrvermerk zu verzichten.

Besprechen Sie diese Problematik mit Ihrer Firma!

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte vorliegt.

Inhaltsverzeichnis

Tabellenverzeichnis	v
Abkürzungsverzeichnis	vi
Kurzfassung (Abstract)	vii
1 Motivation und Zielsetzung	1
1.1 Motivation	1
1.2 Zielsetzung	1
2 Voraussetzungen	2
2.1 Grundlagen der Mathematik	2
2.1.1 Notation	2
2.1.2 Partielle Ableitung	2
2.1.3 Kettenregel	2
2.1.4 Gradient	3
2.1.5 Jacobi-Matrix	3
2.1.6 Hadamard-Produkt	3
2.2 Grundlagen des Maschinellen Lernens	4
2.2.1 Datenvorbereitung	4
2.2.2 Regression	4
2.2.3 Neuronale Netze	4
2.2.4 Verlustfunktion	5
2.2.5 Backpropagation	5
3 Hinleitung	6
3.1 Probleme von Feedforward Neuronalen Netzen	6
3.2 Rekurrente Neuronale Netze als Lösung	6
4 Grundlagen von Rekurrenten Neuronalen Netzen	7
4.1 Einführung in Rekurrente Neuronale Netze	7
4.2 Mathematisches Modell einfacher RNN	7
4.3 Rückwärtspropagation in der Zeit (BPTT)	8
5 Probleme einfacher Rekurrenter Neuronaler Netze	9
5.1 Verschwindende Gradienten	9
5.2 Explodierende Gradienten	9
6 Lösungsansätze für sequenzielle Probleme	10
6.1 Gradienten Clipping	10
6.2 LSTM	10
6.3 GRU	11

7 Zusammenfassung	12
7.1 Fazit	12
7.2 Ausblick	12
Literaturverzeichnis	13

Tabellenverzeichnis

2.1	Im Dokument verwendete mathematische Notation	2
-----	---	---

Abkürzungsverzeichnis

BPTT	Backpropagation Through Time
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
NaN	Not a Number
RNN	Recurrent Neural Network
SGD	Stochastic Gradient Descent

Kurzfassung (Abstract)

Diese Projektarbeit gibt einen strukturierten Überblick über rekurrente neuronale Netze und deren Einsatz bei der Verarbeitung sequenzieller Daten. Ausgehend von den mathematischen Grundlagen und zentralen Konzepten des maschinellen Lernens wird zunächst das Grundmodell einfacher rekurrenter Netze beschrieben. Darauf aufbauend wird das Training mittels Backpropagation Through Time (BPTT) erläutert und in den Kontext der allgemeinen Gradientenberechnung in neuronalen Netzen eingeordnet.

Ein Schwerpunkt der Arbeit liegt auf den typischen Schwierigkeiten beim Training rekurrenter Architekturen. Insbesondere werden die Ursachen und Auswirkungen verschwindender sowie explodierender Gradienten analysiert. Als zentrale Gegenmaßnahmen werden Gradienten-Clipping sowie die erweiterten Architekturen Long Short-Term Memory (LSTM) und Gated Recurrent Unit (GRU) vorgestellt und hinsichtlich ihrer Funktionsweise eingeordnet.

Die Arbeit verfolgt das Ziel, die theoretischen Grundlagen, die praktischen Herausforderungen und die wesentlichen Lösungsansätze im Umfeld rekurrenter neuronaler Netze kompakt und nachvollziehbar darzustellen. Damit schafft sie eine Grundlage für die Bewertung des Einsatzes dieser Modelle in Anwendungsfeldern wie Zeitreihenanalyse und Sprachverarbeitung.

1 Motivation und Zielsetzung

1.1 Motivation

Trotz ihrer theoretischen Stärke leiden einfache Recurrent Neural Networks (RNNs) in der Praxis unter dem Problem verschwindender und explodierender Gradienten, sobald lange Abhängigkeitsketten erlernt werden müssen. Architekturen wie das LSTM und die GRU haben dieses Problem weitgehend abgeschwächt und den Einsatzbereich rekurrenter Modelle erheblich erweitert.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Projektarbeit ist es, RNNs als Klasse sequenzieller Lernmodelle theoretisch fundiert aufzuarbeiten und ihre praktischen Eigenschaften anhand von konkreten Beispielen zu veranschaulichen.

Hieraus leiten sich die folgenden Teilziele ab:

1. **Theoretische Grundlagen:** Darstellung des mathematischen Modells einfacher rekurrenter Netze, einschließlich der Vorwärtsberechnung und der zeitlich entfalteten Rückwärtspropagation mittels BPTT.
2. **Gradientenproblematik:** Analyse der Ursachen verschwindender und explodierender Gradienten sowie Diskussion der wichtigsten Gegenmaßnahmen.
3. **Erweiterte Architekturen:** Vorstellung von LSTM und GRU als Lösungsansätze für das Kurzzeitgedächtnis-Problem einfacher RNNs.
4. **Anwendungsfelder:** Einordnung typischer Einsatzgebiete rekurrenter Netze, insbesondere in der Sprachverarbeitung und Zeitreihenanalyse, um die erarbeiteten Konzepte an konkreten Problemklassen zu veranschaulichen.
5. **Kritische Einordnung:** Reflexion der Grenzen und Stärken rekurrenter Netze im Vergleich zu modernen Alternativen wie *Transformer*-Architekturen.

2 Voraussetzungen

2.1 Grundlagen der Mathematik

2.1.1 Notation

Symbol	Bedeutung
x	Skalar (kursiver Kleinbuchstabe)
$\mathbf{x}$	Vektor (fetter Kleinbuchstabe, Spaltenvektor)
$\mathbf{x}_i$	Vektorelement (i-tes Element)
$\mathbf{x}^\top$	Transponierter Zeilenvektor
$\mathbf{W}$	Matrix (fetter Großbuchstabe)
$W_{i,j}$	Element in Zeile i , Spalte j von $\mathbf{W}$
$\mathbb{R}$	Menge der reellen Zahlen
$\mathbb{R}^n$	n -dimensionaler reeller Vektorraum
$\odot$	Hadamard-Produkt (elementweise Multiplikation)

Tabelle 2.1: Im Dokument verwendete mathematische Notation

2.1.2 Partielle Ableitung

Gegeben eine Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$, die von mehreren Variablen $x_1, x_2, \dots, x_n$ abhängt, beschreibt die partielle Ableitung

$$\frac{\partial f}{\partial x_i} \quad (2.1)$$

die Änderungsrate von f bezüglich x_i , während alle anderen Variablen konstant gehalten werden. Partielle Ableitungen sind die Grundlage der mehrdimensionalen Differentialrechnung und bilden das mathematische Fundament für Optimierungsverfahren im maschinellen Lernen.

2.1.3 Kettenregel

Die Kettenregel beschreibt, wie die Ableitung einer zusammengesetzten Funktion berechnet wird. Sind f und g differenzierbare Funktionen mit $h(x) = f(g(x))$, so gilt:

$$\frac{dh}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx} \quad (2.2)$$

2.1.4 Gradient

Der Gradient einer skalaren Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ist der Vektor aller partiellen Ableitungen:

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} \quad (2.3)$$

Er zeigt in die Richtung des steilsten Anstiegs von f am Punkt $\mathbf{x}$.

2.1.5 Jacobi-Matrix

Die Jacobi-Matrix verallgemeinert den Gradienten auf Funktionen mit Vektoren. Gegeben eine Funktion $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, die einen Eingangsvektor $\mathbf{x} \in \mathbb{R}^n$ auf einen Ausgangsvektor $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^m$ abbildet, ist die Jacobi-Matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ definiert als:

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}. \quad (2.4)$$

Das Element $J_{ij} = \frac{\partial f_i}{\partial x_j}$ gibt an, wie stark sich die i -te Ausgabekomponente ändert, wenn die j -te Eingabekomponente variiert wird.

Für den Spezialfall $m = 1$ reduziert sich die Jacobi-Matrix zum transponierten Gradienten: $\mathbf{J} = \nabla f^\top$.

2.1.6 Hadamard-Produkt

Das Hadamard-Produkt, symbolisiert durch $\odot$, bezeichnet die elementweise Multiplikation von zwei Matrizen oder Vektoren gleicher Dimension. Gegeben zwei Matrizen $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$, ist ihr Hadamard-Produkt $\mathbf{C} = \mathbf{A} \odot \mathbf{B}$ eine Matrix aus $\mathbb{R}^{m \times n}$, deren Elemente wie folgt definiert sind:

$$C_{i,j} = A_{i,j} \cdot B_{i,j} \quad (2.5)$$

Für Vektoren $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ berechnet sich das Hadamard-Produkt äquivalent als $c_i = a_i \cdot b_i$.

2.2 Grundlagen des Maschinellen Lernens

2.2.1 Datenvorbereitung

Bevor ein Modell trainiert werden kann, müssen die Rohdaten in eine geeignete Form gebracht werden. Dieser Schritt wird als Datenvorbereitung bezeichnet und umfasst mehrere Teilaufgaben.

Zunächst werden fehlende Werte behandelt, etwa durch Entfernen betroffener Einträge oder durch das Einsetzen von statistischen Kennzahlen wie dem Mittelwert. Anschließend erfolgt häufig eine Normalisierung oder Standardisierung der Eingabemerkmale, damit numerisch sehr unterschiedliche Größen den Trainingsprozess nicht verschlechtern. Bei der Standardisierung wird jedes Merkmal x transformiert zu

$$x' = \frac{x - \mu}{\sigma}, \quad (2.6)$$

wobei μ der Mittelwert und σ die Standardabweichung des jeweiligen Merkmals sind.

Schließlich wird der Datensatz in mindestens zwei disjunkte Teilmengen aufgeteilt: einen Trainings- und einen Testdatensatz. Optional wird ein dritter Validierungsdatensatz abgetrennt, der zur Hyperparameterauswahl während des Trainings genutzt wird.

2.2.2 Regression

Regression bezeichnet statistische Methoden zur Modellierung der Beziehung zwischen einer Zielgröße und einer oder mehreren unabhängigen Merkmalen. Das einfachste Modell ist die lineare Regression, die eine lineare Abbildung von der Eingabe $\mathbf{x} \in \mathbb{R}^n$ auf eine vorhergesagte Ausgabe $\hat{y} \in \mathbb{R}$ annimmt:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad (2.7)$$

wobei $\mathbf{w}$ der Gewichtsvektor und b der Bias-Term sind.

Die Qualität der Vorhersage wird durch eine Verlustfunktion $\mathcal{L}$ gemessen. Das Ziel des Lernalgorithmus ist es, die Parameter $\mathbf{w}$ und b so zu wählen, dass $\mathcal{L}$ minimiert wird.

2.2.3 Neuronale Netze

Ein künstliches neuronales Netz besteht aus mehreren hintereinandergeschalteten Schichten von sog. Neuronen. Jedes Neuron berechnet eine gewichtete Summe seiner Eingaben und wendet darauf eine nichtlineare Aktivierungsfunktion σ an:

$$a = \sigma(\mathbf{w}^\top \mathbf{x} + b). \quad (2.8)$$

Eine Schicht transformiert einen Eingangsvektor $\mathbf{x}$ zu einem Ausgangsvektor $\mathbf{a}$ mittels einer Gewichtsmatrix $\mathbf{W}$ und eines Biasvektors $\mathbf{b}$:

$$\mathbf{a} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}). \quad (2.9)$$

Durch das Hintereinanderschalten mehrerer solcher Schichten entsteht ein *tiefes* Netz (*Deep Neural Network*), das in der Lage ist, hochgradig nichtlineare Zusammenhänge zu erlernen. Das gesamte Netz realisiert damit eine parametrisierte Funktion $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$, deren Parameter $\theta = \{\mathbf{W}^*, \mathbf{b}^*\}$ durch Training bestimmt werden.

2.2.4 Verlustfunktion

Die Verlustfunktion (auch Loss-Funktion oder Kostenfunktion genannt) quantifiziert die Abweichung zwischen der vom Modell vorhergesagten Ausgabe $\hat{y}$ und dem tatsächlichen Zielwert y . Die Minimierung dieser Funktion $\mathcal{L}$ während des Trainingsprozesses ist das Hauptziel von Optimierungsalgorithmen im maschinellen Lernen [5].

Abhängig von der zugrundeliegenden Problemstellung kommen üblicherweise unterschiedliche Verlustfunktionen l zum Einsatz [2].

Die allgemeine Lossfunktion für einen Datensatz mit n Beispielen wird durch den Durchschnitt der Einzelfehler über alle Beispiele definiert:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n l(\hat{\mathbf{y}}_i, \mathbf{y}_i) \quad (2.10)$$

2.2.5 Backpropagation

Der Algorithmus zur Berechnung der Gradienten in neuronalen Netzen wird als Backpropagation bezeichnet. Er wendet die Kettenregel der Differentialrechnung rekursiv an, um die partiellen Ableitungen der Verlustfunktion $\mathcal{L}$ nach allen Parametern effizient zu berechnen.

In der Praxis wird Backpropagation mit einem Gradientenabstiegsverfahren kombiniert. Beim Stochastic Gradient Descent (SGD) werden die Parameter nach jedem Mini-Batch aktualisiert:

$$(\mathbf{W}, \mathbf{b}) \leftarrow (\mathbf{W}, \mathbf{b}) - \eta \nabla_{(\mathbf{W}, \mathbf{b})} \mathcal{L} \quad (2.11)$$

wobei $\eta > 0$ die Lernrate ist.

3 Hinleitung

3.1 Probleme von Feedforward Neuronalen Netzen

Feedforward-Netze erzielen in vielen Anwendungsfeldern gute Ergebnisse, stoßen jedoch bei der Verarbeitung sequenzieller Daten an grundlegende Grenzen. Klassische Feedforward-Netze setzen eine feste Eingabelänge voraus. Reale Daten liegen jedoch häufig in variabler Länge vor, etwa bei Texten, Sprachsignalen oder historischen Kursverläufen. Um solche Daten dennoch zu verarbeiten, sind oft zusätzliche Vorverarbeitungsschritte notwendig, die entweder Information verlieren oder das Modell unnötig kompliziert machen.

Ein weiterer Nachteil ist das fehlende Gedächtnis über frühere Eingaben. Gerade bei Aufgaben, in denen der zeitliche oder logische Kontext entscheidend ist, reicht ein solches Vorgehen nicht aus. Für die Modellierung von Abhängigkeiten innerhalb von Sequenzen werden daher Architekturen benötigt, die Vergangenes berücksichtigen können.

3.2 Rekurrente Neuronale Netze als Lösung

Rekurrente neuronale Netze stellen einen Ansatz dar, um diese Einschränkungen zu überwinden. Im Gegensatz zu Feedforward-Netzen verarbeiten sie Daten nicht nur punktuell, sondern schrittweise entlang einer Sequenz. Dabei wird Information aus vorherigen Verarbeitungsschritten intern weitergegeben, sodass frühere Eingaben in die Bewertung aktueller Eingaben einfließen können.

Dadurch eignen sich RNNs insbesondere für Aufgaben, bei denen Reihenfolge und Kontext eine wichtige Rolle spielen. Beispiele hierfür sind die Analyse von Zeitreihen, die Verarbeitung natürlicher Sprache oder die Vorhersage von Entwicklungen auf Basis historischer Daten. Ein wesentlicher Vorteil besteht darin, dass Sequenzen variabler Länge verarbeitet werden können, ohne dass jede Eingabe auf dieselbe feste Struktur gebracht werden muss.

4 Grundlagen von Rekurrenten Neuronalen Netzen

4.1 Einführung in Rekurrente Neuronale Netze

Rekurrente neuronale Netze (RNN) sind eine spezielle Architektur von neuronalen Netzen, die primär für die Verarbeitung von sequenziellen Daten entwickelt wurden. Im Gegensatz zu klassischen Feedforward-Netzen, bei denen der Informationsfluss streng in eine Richtung von der Eingabe- zur Ausgabeschicht verläuft, besitzen diese Modelle Rückkopplungen in Form von internen Schleifen [11]. Diese Schleifen ermöglichen es dem Netzwerk, Informationen über vorherige Zeitschritte hinweg in einem sogenannten verborgenen Zustand zu speichern. Die Grundlagen dieses Modells wurden unter anderem maßgeblich durch die Arbeiten von Elman geprägt [4]. Dadurch können RNNs zeitliche oder sequentielle Abhängigkeiten modellieren, was sie besonders nützlich für Anwendungen wie Sprachverarbeitung, Zeitreihenanalyse und maschinelle Übersetzung macht.

4.2 Mathematisches Modell einfacher RNN

Die Funktionsweise eines einfachen RNN lässt sich durch eine Reihe mathematischer Gleichungen beschreiben, die in jedem Zeitschritt t ausgeführt werden. Die Eingabe zum Zeitpunkt t sei ein Vektor $\mathbf{x}_t \in \mathbb{R}^x$. Der verborgene Zustand $\mathbf{h}_t \in \mathbb{R}^h$ wird berechnet, indem sowohl die aktuelle Eingabe als auch der vorherige verborgene Zustand $\mathbf{h}_{t-1}$ einbezogen werden. Die Gleichung lautet:

$$\mathbf{h}_t = \sigma(\mathbf{W}_{h,x}\mathbf{x}_t + \mathbf{W}_{h,h}\mathbf{h}_{t-1} + \mathbf{b}_h) \quad (4.1)$$

Hierbei sind $\mathbf{W}_{h,x}$ und $\mathbf{W}_{h,h}$ die Gewichtsmatrizen für die Eingabe beziehungsweise den vorherigen verborgenen Zustand, und $\mathbf{b}_h$ ist ein Bias-Vektor. Die Funktion σ ist eine nichtlineare Aktivierungsfunktion, typischerweise der Tangens hyperbolicus ($\tanh$) oder die Sigmoid-Funktion [11].

Die Ausgabe $\mathbf{o}_t \in \mathbb{R}^o$ im Zeitschritt t wird oft durch eine separate Transformation des aktuellen verborgenen Zustands berechnet:

$$\mathbf{o}_t = \phi(\mathbf{W}_{o,h}\mathbf{h}_t + \mathbf{b}_o) \quad (4.2)$$

Dabei ist $\mathbf{W}_{o,h}$ die Gewichtsmatrix für die Ausgabe, $\mathbf{b}_o$ der Ausgabe-Biasvektor und ϕ die Ausgabe-Aktivierungsfunktion (z. B. Softmax für Klassifikationsaufgaben). Bemerkenswert ist, dass die Gewichtsmatrizen ($\mathbf{W}_{h,x}$, $\mathbf{W}_{h,h}$, $\mathbf{W}_{o,h}$) über alle Zeitschritte hinweg geteilt werden, was die Anzahl der zu lernenden Parameter deutlich reduziert.

4.3 Rückwärtspropagation in der Zeit (BPTT)

Das Training von RNN erfolgt typischerweise mittels einer spezialisierten Form der Backpropagation, die als BPTT bezeichnet wird [11]. Die zugrundeliegende Idee der generellen Fehler-Backpropagation wurde insbesondere durch die Publikationen von Rumelhart et al. etabliert [9], während das BPTT-Verfahren von Werbos detailliert für rekurrente Netzwerke mathematisch hergeleitet und formuliert wurde [10]. Bei BPTT wird das rekurrente Netz über die gesamte Länge der Sequenz entfaltet (unrolled). Das entfaltete Netz kann man sich als ein tiefes Feedforward-Netz vorstellen, bei dem jede Schicht einem definierten Zeitschritt entspricht.

Nachdem in einem Vorwärtsthroughlauf alle Zustände und Ausgaben berechnet wurden, wandert der Fehlergradient beim BPTT-Verfahren von den späten Zeitschritten rückwärts durch die Zeit zu den früheren Zeitschritten.

Die allgemeine Lossfunktion über eine Sequenz von Länge T wird durch den Durchschnitt der Einzelfehler über alle Zeitschritte definiert:

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^T l(\mathbf{o}_t, \mathbf{y}_t) \quad (4.3)$$

Die einzelnen Parameter des Netzwerks (z. B. $\mathbf{W}_{h,x}$, $\mathbf{W}_{h,h}$, $\mathbf{W}_{o,h}$) werden dann durch die Kettenregel der Differenzialrechnung aktualisiert, um die Lossfunktion zu minimieren.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{o}_t} = \frac{\partial l(\mathbf{o}_t, \mathbf{y}_t)}{T \cdot \partial \mathbf{o}_t} \quad (4.4)$$

5 Probleme einfacher Rekurrenter Neuronaler Netze

Beim Training von künstlichen neuronalen Netzen mittels BPTT wird der Fehlergradient von der Ausgabeschicht durch die Zeit in die Vergangenheit zurückgeführt, um die Gewichte des Netzes anzupassen. Bei einfachen rekurrenten Netzen führt dies bei langen Eingabesequenzen zu erheblichen Schwierigkeiten beim Erlernen von weitreichenden Abhängigkeiten [1].

5.1 Verschwindende Gradienten

Das Problem der verschwindenden Gradienten (engl. *Vanishing Gradients*) tritt auf, wenn die Ableitungen der Aktivierungsfunktionen sowie die Gewichte in der rekurrenten Schicht klein sind. Bei der BPTT wird der Gradient mit jedem Zeitschritt, den er in die Vergangenheit zurückfließt, durch Kettenregel-Multiplikationen skaliert. Sind diese Faktoren überwiegend kleiner als eins, schrumpft der Gradient exponentiell mit der Sequenzlänge. Dies hat zur Folge, dass das Netz keine weitreichenden zeitlichen Abhängigkeiten erlernen kann, da der Beitrag weit zurückliegender Eingaben zur Fehlerminimierung nahezu null wird [6, 1].

5.2 Explodierende Gradienten

Im Gegensatz dazu kommt es zum Problem der explodierenden Gradienten (engl. *Exploding Gradients*), wenn die Gewichtsmatrizen große Eigenwerte besitzen. Die wiederholte Multiplikation der Gradienten über viele Zeitschritte führt in diesem Fall zu einem exponentiellen Wachstum. Dies verursacht extrem große und unkontrollierte Aktualisierungen der Gewichte während des Optimierungsprozesses, was das Training stark destabilisiert und im schlimmsten Fall zu numerischen Überläufen (Not a Number (NaN)) führt [8].

6 Lösungsansätze für sequenzielle Probleme

6.1 Gradienten Clipping

Ein pragmatischer und weit verbreiteter Ansatz zur Lösung des Problems der explodierenden Gradienten ist *Gradienten-Beschneidung* (*Gradient Clipping*). Wie von Pascanu et al. [8] detailliert analysiert, kann der Gradient während des Trainings von rekurrenten Netzwerken durch BPTT exponentiell anwachsen. Um die daraus resultierende Instabilität der Gewichtsaktualisierungen zu verhindern, wird der Gradientenvektor künstlich skaliert, sobald seine Norm einen vordefinierten Schwellenwert überschreitet.

Mathematisch lässt sich das Gradienten Clipping über die L_2 -Norm des Gradienten $\mathbf{g}$ wie folgt formulieren:

$$\mathbf{g} \leftarrow \begin{cases} \frac{c}{\|\mathbf{g}\|} \mathbf{g} & \text{falls } \|\mathbf{g}\| > c \\ \mathbf{g} & \text{sonst,} \end{cases} \quad (6.1)$$

wobei c der gewählte Schwellenwert (Threshold) ist. Diese Methode stellt sicher, dass die Richtung des Gradienten im Parameterraum erhalten bleibt, während seine maximale Schrittweite auf einen sicheren Wert begrenzt wird.

6.2 LSTM

Das LSTM ist die bekannteste und am weitesten verbreitete Architektur zur Bewältigung des Problems der verschwindenden Gradienten. Sie wurde ursprünglich von Hochreiter und Schmidhuber [7] entwickelt. Kern des Modells ist ein interner Zellzustand (Cell State), dessen Informationsfluss durch drei verschiedene Gates – *Forget Gate*, *Input Gate* und *Output Gate* – reguliert wird. Dies ermöglicht einen konstanten Fehlerfluss durch die Zeit, wodurch das Netzwerk auch über lange Sequenzen hinweg stabil trainiert werden kann.

Die mathematische Funktionsweise einer LSTM-Zelle in einem Zeitschritt t definiert sich durch folgende Gleichungen:

$$\mathbf{I}_t = \sigma(\mathbf{X}_t \mathbf{W}_{x,i} + \mathbf{H}_{t-1} \mathbf{W}_{h,i} + \mathbf{b}_i) \quad (\text{Input Gate}) \quad (6.2)$$

$$\mathbf{F}_t = \sigma(\mathbf{X}_t \mathbf{W}_{x,f} + \mathbf{H}_{t-1} \mathbf{W}_{h,f} + \mathbf{b}_f) \quad (\text{Forget Gate}) \quad (6.3)$$

$$\mathbf{O}_t = \sigma(\mathbf{X}_t \mathbf{W}_{x,o} + \mathbf{H}_{t-1} \mathbf{W}_{h,o} + \mathbf{b}_o) \quad (\text{Output Gate}) \quad (6.4)$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{x,c} + \mathbf{H}_{t-1} \mathbf{W}_{h,c} + \mathbf{b}_c) \quad (\text{Kandidat für Zellzustand}) \quad (6.5)$$

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \quad (\text{Zellzustand}) \quad (6.6)$$

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \quad (\text{Verborgener Zustand}) \quad (6.7)$$

Durch das Zusammenspiel aus Input- und Forget-Gate kann das LSTM alte Informationen gezielt verlernen und neue systematisch in den globalen Zellzustand $\mathbf{C}_t$ einspeichern.

6.3 GRU

Die GRU wurde von Cho et al. [3] als eine vereinfachte, aber leistungsstarke Alternative zum LSTM vorgestellt. Das Hauptziel der GRU ist es ebenfalls, das Problem der verschwindenden Gradienten zu umgehen und das Erlernen weitreichender Abhängigkeiten in Sequenzen zu erleichtern, jedoch mit einer reduzierten Anzahl an Parametern.

Eine GRU erreicht dies durch die Verwendung von zwei Steuereinheiten (Gates): dem *Update Gate* und dem *Reset Gate*. Diese Gates regulieren den reibungslosen Informationsfluss und entscheiden, welche Informationen beibehalten und welche verworfen werden. Die mathematische Umsetzung einer GRU-Zelle im Zeitschritt t für eine Eingabe $\mathbf{x}_t$ und den verborgenen Zustand $\mathbf{h}_{t-1}$ lautet wie folgt:

$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{x,z} + \mathbf{H}_{t-1} \mathbf{W}_{h,z} + \mathbf{b}_z) \quad (\text{Update Gate}) \quad (6.8)$$

$$\mathbf{R}_t = \sigma(\mathbf{X}_t \mathbf{W}_{x,r} + \mathbf{H}_{t-1} \mathbf{W}_{h,r} + \mathbf{b}_r) \quad (\text{Reset Gate}) \quad (6.9)$$

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{x,h} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{h,h} + \mathbf{b}_h) \quad (\text{Kandidat für verborgenen Zustand}) \quad (6.10)$$

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t \quad (\text{Neuer verborgener Zustand}) \quad (6.11)$$

7 Zusammenfassung

7.1 Fazit

Die Arbeit hat gezeigt, dass RNNs eine leistungsfähige Modellklasse für die Verarbeitung sequenzieller Daten darstellen. Ihr zentraler Vorteil liegt darin, dass frühere Eingaben über verborgene Zustände in die Verarbeitung späterer Zeitschritte einbezogen werden können. Gleichzeitig wurde deutlich, dass einfache rekurrente Architekturen beim Training auf langen Sequenzen unter verschwindenden und explodierenden Gradienten leiden.

Zur Bewältigung dieser Schwierigkeiten wurden mit Gradienten-Clipping, LSTM und GRU wesentliche Lösungsansätze herausgearbeitet. Insbesondere LSTM und GRU erweitern das Grundmodell um Kontrollmechanismen, die den Informationsfluss stabilisieren und dadurch das Lernen längerer Abhängigkeiten verbessern. Damit bleibt der Einsatz rekurrenter Netze trotz moderner Alternativen in vielen Anwendungsbereichen weiterhin fachlich relevant.

7.2 Ausblick

Für zukünftige Arbeiten bietet sich eine weiterführende Gegenüberstellung rekurrenter Architekturen mit modernen sequenziellen Modellen wie Transformern an. Darüber hinaus wäre eine empirische Evaluation auf realen Datensätzen sinnvoll, um die theoretisch beschriebenen Unterschiede zwischen einfachen RNNs, LSTM und GRU anhand von Trainingsstabilität, Modellgüte und Rechenaufwand systematisch zu vergleichen.

Literaturverzeichnis

- [1] Yoshua Bengio, Patrice Simard und Paolo Frasconi. „Learning long-term dependencies with gradient descent is difficult“. In: *IEEE Transactions on Neural Networks* 5.2 (1994), S. 157–166. DOI: 10.1109/72.279181.
- [2] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. ISBN: 978-0387310732.
- [3] Kyunghyun Cho et al. „Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation“. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2014), S. 1724–1734. DOI: 10.3115/v1/D14-1179.
- [4] Jeffrey L. Elman. „Finding structure in time“. In: *Cognitive Science* 14.2 (1990), S. 179–211. DOI: 10.1207/s15516709cog1402_1.
- [5] Ian Goodfellow, Yoshua Bengio und Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: <http://www.deeplearningbook.org>.
- [6] Sepp Hochreiter. „Untersuchungen zu dynamischen neuronalen Netzen“. Diploma thesis. Technische Universität München, 1991. URL: <https://www.bioinf.jku.at/publications/older/ch7.pdf>.
- [7] Sepp Hochreiter und Jürgen Schmidhuber. „Long Short-Term Memory“. In: *Neural Computation* 9.8 (1997), S. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [8] Razvan Pascanu, Tomas Mikolov und Yoshua Bengio. „On the difficulty of training recurrent neural networks“. In: *International Conference on Machine Learning (ICML)*. PMLR, 2013, S. 1310–1318. URL: <https://proceedings.mlr.press/v28/pascanu13.html>.
- [9] David E. Rumelhart, Geoffrey E. Hinton und Ronald J. Williams. „Learning representations by back-propagating errors“. In: *Nature* 323.6088 (1986), S. 533–536. DOI: 10.1038/323533a0.
- [10] Paul J. Werbos. „Backpropagation through time: what it does and how to do it“. In: *Proceedings of the IEEE* 78.10 (1990), S. 1550–1560. DOI: 10.1109/5.58337.
- [11] Aston Zhang et al. *Dive into Deep Learning*. Cambridge University Press, 2024. ISBN: 978-1-009-38943-3.